

La Cave du Coin — Récapitulatif de la séance du 15/09/2026

Ce qui a été fait aujourd'hui, dans l'ordre, pour pouvoir s'y retrouver plus tard.

1. Lancement du projet en local

Docker n'étant pas installé sur ce PC, l'application a été lancée en mode "local classique" (environnement virtuel Python + base SQLite), le chemin déjà validé dans le README.

Commande / action	Ce que ça fait et pourquoi
<code>./venv/Scripts/python.exe -m alembic upgrade head</code>	Applique les migrations de schéma (crée/met à jour les tables) sur la base SQLite locale.
<code>./venv/Scripts/python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload</code>	Démarre le serveur web local. "--reload" relance automatiquement le serveur si le code change. Accessible ensuite sur <code>http://127.0.0.1:8000</code> (interface) et <code>/docs</code> (API).

Pour arrêter : Ctrl+C dans le terminal où le serveur tourne.

2. Schéma relationnel de la base

Un schéma relationnel complet (12 tables : users, products, sales, cash_sessions, customers...) a été extrait du code (`app/models.py`) et écrit dans `schema.sql` à la racine du projet — un DDL PostgreSQL de référence, purement documentaire (les vraies tables sont créées par Alembic, pas par ce fichier).

3. Réflexion sur le "nettoyage" de la base de prod

Un script existant, `scripts/reset_prod_database.sh`, permettrait de vider entièrement la prod (sauvegarde automatique avant, puis TRUNCATE de toutes les tables, puis recréation des seuls comptes par défaut). **Décision : ne pas l'exécuter pour l'instant** — une seule ligne "parasite" à zéro a été repérée dans l'historique des sessions de caisse du compte manager, jugée sans conséquence. Elle peut être supprimée plus tard avec un simple `DELETE FROM cash_sessions WHERE id = <id>;` ciblé si besoin, plutôt qu'un reset complet.

4. Accès pgAdmin à la vraie base de production

C'est le morceau principal de la séance. pgAdmin existait déjà (mis en place lors d'une séance précédente, cf. `Mise_en_place_environnement_staging.pdf`) mais ne voyait que la base de **staging**. Objectif : qu'il puisse aussi interroger la vraie prod, sans jamais avoir besoin de SSH ou de taper du SQL brut dans la console.

5.1 — Modification du code (faite ici, sur ce PC)

Dans `docker-compose.yml`, le service db de la prod a été ajouté au réseau Docker partagé `caddy-net` (déjà utilisé par Caddy et pgAdmin), sous l'alias `proddb` — un nom différent du db de staging pour éviter toute collision. Il reste aussi sur son réseau habituel, donc l'application prod continue de le joindre normalement. Commit `f41df95`, poussé sur GitHub (branche main).

5.2 — Application sur le serveur (VPS)

Commande / action	Ce que ça fait et pourquoi
<code>git pull</code>	Récupère la modification de <code>docker-compose.yml</code> depuis GitHub.

<code>./scripts/add_pgadmin_domain.sh</code>	Script déjà présent qui ajoute PGADMIN_DOMAIN au .env de prod sans risquer une faute de frappe sur le "_". Ici, déjà fait lors d'une séance précédente ("rien à faire").
<code>docker compose up -d --build</code>	Relance le stack prod (db, app, caddy) en tenant compte du nouveau réseau — db rejoint caddy-net sous l'alias proddb.
<code>docker compose -p staging --env-file .env.staging -f docker-compose.yml -f docker-compose.staging.yml up -d</code>	Relance aussi le stack staging par sécurité, pour confirmer que tout communique bien.

5.3 — Incident : compte pgAdmin verrouillé

En essayant de se connecter à pgAdmin, le compte s'est retrouvé verrouillé (trop de tentatives avec un mot de passe qui ne correspondait plus). Comme pgAdmin ne crée son compte admin qu'au tout premier démarrage de son volume de données, la solution a été de repartir d'un volume neuf :

Commande / action	Ce que ça fait et pourquoi
<code>docker volume ls</code>	Liste tous les volumes Docker pour repérer le nom exact de celui de pgAdmin (staging_pgadmin_data).
<code>docker compose -p staging ... stop pgadmin</code>	Arrête le conteneur pgAdmin (sans le supprimer, ce qui viendra ensuite).
<code>docker compose -p staging ... rm -f pgadmin</code>	Supprime le conteneur arrêté — nécessaire car Docker refuse de supprimer un volume tant qu'un conteneur (même arrêté) y fait encore référence.
<code>docker volume rm staging_pgadmin_data</code>	Supprime le volume : pgAdmin va tout recréer de zéro (compte admin inclus) au prochain démarrage, avec les identifiants actuels de .env.staging.
<code>docker compose -p staging ... up -d pgadmin</code>	Redémarre pgAdmin sur un volume neuf → nouveau compte admin fonctionnel.

5.4 — Enregistrement des deux bases dans pgAdmin

Depuis le navigateur (<https://pgadmin.lacaveducoin.com>), "Register → Server" a été fait deux fois :

- **Serveur "Prod"** : host proddb, port 5432, base/utilisateur cave_du_coin, mot de passe = POSTGRES_PASSWORD du .env de production.
- **Serveur "Staging"** : host db (même stack que pgAdmin, pas besoin d'alias), même port/base/utilisateur, mot de passe = celui de .env.staging.

Résultat : les deux mots de passe générés (POSTGRES_PASSWORD) se trouvent être des chaînes hexadécimales pures (chiffres et lettres a-f uniquement) — donc aucun caractère à risque, lisibles et retapables directement à l'écran de la console sans aucun contournement.

5. Le bug clavier de la console Hetzner (rappel)

La console web du VPS ignore systématiquement la touche **Shift** à la frappe (et au collage), quel que soit le clavier physique (AZERTY ou QWERTY) — ce n'est pas une question de touche, c'est la console elle-même qui ne transmet pas le modificateur Shift. Concrètement : `_` devient `-`, `|` devient `\`, etc.

Astuce découverte aujourd'hui pour taper un underscore malgré tout, sans utiliser Shift : construire le caractère via son code octal (95 = `_`), avec des caractères tous confirmés "sûrs" (lettres, chiffres, guillemet simple, backtick) :

Commande / action	Ce que ça fait et pourquoi
<code>`printf '\137'`</code>	S'exécute et produit littéralement un "_" à l'endroit où c'est inséré. Exemple complet : <code>docker compose exec db psql -U cave`printf '\137'`du`printf '\137'`coin -d cave`printf '\137'`du`printf '\137'`coin</code>
Touche Tab (autocomplétion)	Une fois dans psql, taper un début de nom (ex: "cash") puis Tab complète le reste ("cash_sessions") sans jamais appuyer sur la touche _ — marche aussi pour les noms de fichiers/dossiers dans le shell.
<code>echo test_test</code>	Test rapide à faire avant toute commande importante contenant un "_", pour vérifier si le bug est actif ce jour-là (résultat attendu si bug : test-test ou testtest).

Si la commande à taper reste trop complexe malgré ces astuces (beaucoup de "_", "@", "("...), le plus fiable reste un vrai terminal SSH depuis un autre appareil/réseau (téléphone en partage de connexion, PC personnel) — la console Hetzner n'est qu'un filet de secours quand SSH direct n'est pas disponible.

6. Où retrouver les identifiants

Commande / action	Ce que ça fait et pourquoi
<code>grep POSTGRES .env</code>	Sur le VPS, depuis /opt/La_Cave_du_Coin : mot de passe de la vraie base de prod (POSTGRES_PASSWORD).
<code>grep POSTGRES .env.staging</code>	Mot de passe de la base de staging.
<code>grep PGADMIN .env.staging</code>	Email et mot de passe pour se connecter à l'interface pgAdmin elle-même.

Aucun de ces fichiers n'est sur GitHub (exclus via .gitignore) — ils n'existent que sur le VPS.

7. Ce qui reste à faire

- Changer les mots de passe par défaut des comptes de démo (admin/Admin123!, manager/Manager123!, caissier/Caissier123!) avant un vrai usage commercial.
- Vérifier que PGADMIN_DEFAULT_PASSWORD n'est plus la valeur par défaut ChangeMePgAdmin123 — pgAdmin donnant maintenant accès à la vraie prod, un mot de passe faible ici expose directement la production.
- Vérifier/révoquer le jeton GitHub (PAT) créé pour cloner le dépôt privé sur le VPS, s'il n'est plus utile.